DBMS

Data → Collection of facts & figures that can produce meaning ful information.

DBMS → Data stored in structured way so access manager easily.

File Syste → A method of storing managing ~~file~~ in data in files.

Data redandancy and inconsistent
Data isolation and security

Touple row in DB Attribute: A Colourmn

E R Entity represent, (attribute relation)

Normali... Organizing database tables. to reduce data redundancy and anomalies

Redundancy Unnecessary duplication of data
Anomali Problem in insert, update, delete.

Atomic Value A single indivisible value in table
Candidate Attribute that (set or single) that identify.

Transitively A determines B determines C, A indirect to C

Determinant A or set of Attribute that determines another
Non trivial
multivalued When a attribute determines multiple independent values of another attribute.

17 Oct 2026, 4:00 pm

Date

1NF	Each cell contains a single, atomic value
2NF	In 1NF, and every non key attribute fully depends on the whole candidate key.
3NF	In 2NF, and non key attribute transitively depends on a candidate key.
BCNF	Every determinant must be a superkey.
4NF	In BCNF, and no non-trivial multivalued dependency exists unless its determinant is a superkey.
5NF	Every non-trivial join dependency is implied by candidate key.

1NF Paras - Java, DSA → Paras-Java, Paras-DSA

2NF before

Stuid	SubID	Name
1	DB01	Paras
1	JV01	Paras

StuId	Name
1	Paras

Stuid	Subid
1	DB01
1	JV01

17 Oct 2026, 4:05 pm

III NF Before

Stuid	Depid	DeptName
1	D01	CSE
2	D01	CSE

After

Suid	Dept_id
1	D01
2	D01

Dept_id	Dept_Name
D01	CSE
D02	AI

BCNF Boyce Codd Normal Form

Stu	Sub	Instructor
Paras	DSA	Sayan
Rahul	DSA	Sayan
Jain	Java	Seema
Naren	DSA	Ram

Instructor is not superkey because one instructor can teach multiple student.

17 Oct 2026, 4:05 pm

IV NF A studen can have multiple hobbies and language

Name	Hobby	Language
Paras	Cricket	Hindi
Paras	Music	English
Paras	Music	Hindi
Paras	Cricket	English.

Name	Hobby
Paras	Cricket
Paras	Cricket

Name	Language
Paras	Hindi
Paras	English.

5 NF Must be in 4NF and every non trivial dependency must be implied by candidate keys.

Ex. A company assign suppliers to projects to ~~projects to~~ provides specific parts.

Suppose a business rule guarantees that supplier can supply a part to a project whenever all three pairwise relationships exist.

instead Supplier_P_P (Supplier, Project, Part)

(3) →

Supplier_project (Supplier, Project)
Supplier_Part (supplier, Part)
Project_Part (Project, Part)

17 Oct 2026, 4:05 pm

Page No.
Date

Decomposition - Splitting one large database table into smaller tables to reduce redundancy and anomalies.

Lossless Join Decomposition: splitting a table into smaller tables so that joining them back reconstructs the original table exactly, without loosing or adding any rows.

Serializability when multiple transaction run simultaneously, the final result should be the same as if they run one after another

T1 Add 100 let initial 1000
T2 Subtract 50 final would be 950

Conflict Serializability - A schedule can be converted into a serial serial schedule by swapping non conflicting.
→ if we can rearrange non-conflicting operations to make transaction run one after another it is CS.

View Serializability: If transaction read the same values and the final write is done by the same transaction as in serial execution it is view serializable.

17 Oct 2026, 4:05 pm

ACID

Atomicity: A transaction happens completely or not at all.
Why: To prevent incomplete transaction
Eg.: Money transfer fails → money is not deducted from the sender.

Consistency: A transaction keeps the database valid according to its rules
Why: To maintain data correctness
Eg.: A money transfer of 500 from 1000 leaves the sender -500 and the receiver with 500+

Isolation: Concurrent transaction should not interfere with each other incorrectly.
Why: To prevent incorrect results when transaction run together
Eg.: Two people booking last seat should not both get.

Durability: Once a transaction is commited, its changes are saved even after system crash.
Why: to prevent commited data from being lost
Eg.: After successful money transfer, the updated balance remains saved even if the server restarts.

17 Oct 2026, 4:05 pm

Concurrency Control

: Managing multiple transaction running simultaneously

Why : To prevent incorrect data and maintain consistency

Example → Two people try to book last seat only one should get it.

Locking : A machanism that restricts access to data while transaction uses it.

Why: To prevent conflicting transactions.

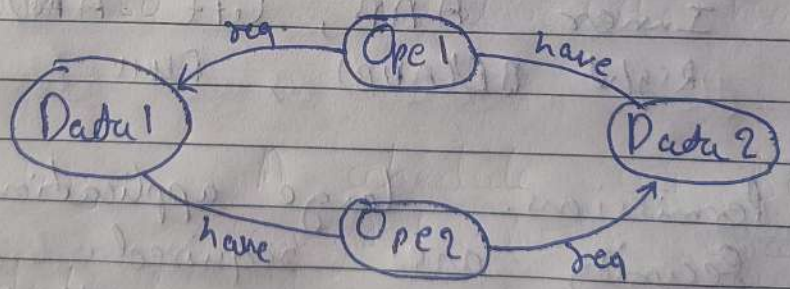
Two Phase locking : A locking protocol with two phase.

1) Growing phase: Acquire lock
2) Release (shrinking): Release lock.

Why to ensure Conflict serializability

Eg. T1 lock an account, update balance, then release the lock t2 can then access it.

Deadlock



Prevention : Ensuring transaction cannot enter a circular waiting situation (should have order)

Detection : If graph shows $T1 \rightarrow T2 \rightarrow T_i$, a deadlock exists; the DBMS can abort one transaction to resolve it

17 Oct 2026, 4:06 pm

DBMS Commands

SQL — DDL, DML, DQL, DCL, TCL

DDL	Define database Structure	CREATE, ALTER, DROP TRUNCATE
DML	Data Manipulation language	INSERT, UPDATE DELETE
DQL	Retrieves Data	SELECT
DCL	Controls Permission	GRANT, REVOKE
TCL	Transaction Manages	COMMIT, ROLLBACK SAVEPOINT

Join — Inner A ∩ B, Left A (A∪B), Right B (A∪B), full A ∪ B

DCL — Permissions. large application DB level Security are required. to ~~prevent~~ safe from sql injection, hacker attack.

TCL

```
Start transaction;
Update accounts
SET balance = balance - 500 WHERE id = 1;

UPDATE accounts
SET balance = balance + 500 WHERE id = 2
COMMIT;
```

17 Oct 2026, 4:06 pm

How SQL Execute Commands.

SQL query → Parser → Validation → Query Optimizer → Execution Engine → Storage Engine → Result.

Parser → Check Syntax

Validation → table name, column names, permission and other rules.

Optimizer → Choose an efficiend execution plan

Execution → Execute the chosen plan

Storage → Read or modifieds the stored data

Result → Return the result or reports success or failure.

Order of SQL Commands

```
SELECT department, AVG(marks) AS avg_marks
FROM students
WHERE marks > 40
GROUP BY department
HAVING AVG(marks) > 70
ORDER BY avg_marks DESC
LIMIT 5;
```

17 Oct 2026, 4:06 pm

NoSQL Commands Mongoose.

await Student.create({name:"Paras", marks:90})

student = await Student.find();

findOne findById

await Student.updateOne({name:"Paras"}, {$set:{marks:95}});

their {returnDocument:"after"} we can put here

await Student.deleteOne({id:''})

$gt, $lt, $gte, $lte, $ne, $in

find({marks:{$gt:80}});
find().sort({marks:-1});

there is select as well after find(). select("","");

Aggrigation

```
const result = await Student.aggregate([
  {
    $group: {
      _id: null,
      averageMarks: { $avg: "$marks" }
    }
  }
]);
```

17 Oct 2026, 4:06 pm

OLD Notes

17 Oct 2026, 4:07 pm

17 Oct 2026, 4:08 pm

DATA BASE

First thing the DB is the pard of Backend.

It is use to Store data, And Access using node

My Sql → it is a DB

Sql → it is a lunguage.

Mongo db is non relatimal db.

DB → A collection of data that can easily accessed.

Why DB →

- can store large data
- feature like ~~seq~~ security, scalability
- Easier to insert, update or delete data.

DBMS, SQL, are the just upper layer.

17 Oct 2026, 4:08 pm

SQL

Relational Database
(data in (stored in Relation (table))

eg. MySQL, Oracle, postgre SQL

No SQL

Non Relational Database

data stored in the form of document /key-val graph etc.

eg. Mongo DB, Cassandra, Neo4j etc.

17 Oct 2026, 4:08 pm

SQL →
Structured Query language.

A colown in sql table → is a design → named schema

A row in sql table → is named as tuple

My SQL root password: ParasMERN@12

three basic commands is basics

```
CREATE DATABASE college; // to create
DROP DATABASE college; // to delete.
USE college; // to ~~do~~ create table
```

First SQL

① create database apnacollege;
② use apna college
③ Create table student (
rollno INT,
name varchar(30),
age Int
);

insert into student
values
(21, "Paras", 20)
(23, "Parwar", 21);

Select * from Student;

16/3/24
50

Database → Queries if Not

```
CREATE DATABASE db-name;
CREAT E DATABASE If NOT EXISTS db name;

DROP DATA DASE db name;
DROP DATABASE IF EXISTS db-name;

SHOW DATABASE;
SHOW TABLES;
```

SQL →
Datatable
→ Constraints
→ Basic

(1) Not Null,
(2) unique,
(3) default 0,
(4) check
→
constraints age_check check (age >18 AND age <60)

17 Oct 2026, 4:08 pm

Date Page 17/9/24

Key K
Constraints,

PRIMARY KEY →

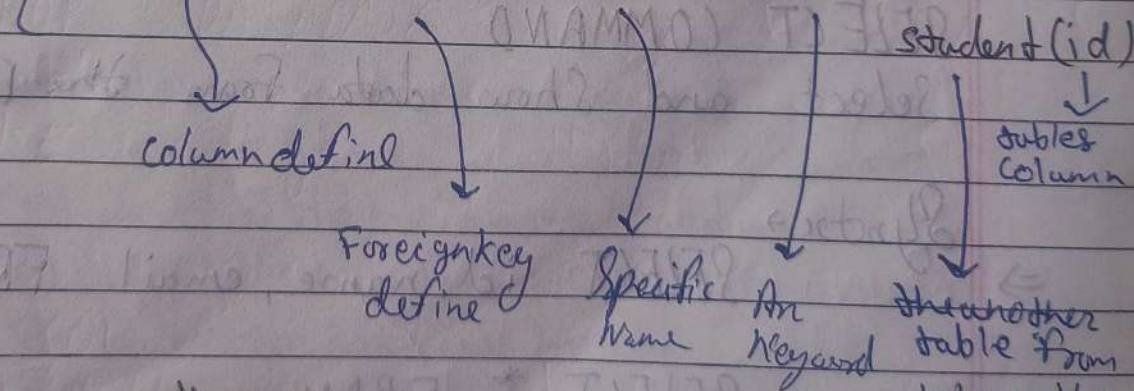
make a colum unique & not null but used only for one.

id INT PRIMARY KEY,
or PRIMARY KEY (id,
Only one column or set of column, unique, Notnull.

FOREIGN KEY →

Prevent actions that would destroy links b/w tables.

cust_id INT,
FOREIGN KEY (cust_id) references student(id),



Can be more then one.

ER Diagram → Database → Reverse Engineering (ctrl+R

Relation. — table.

17 Oct 2026, 4:08 pm

17/3/24
52

INSERT INTO TABLE

Formate →

Use table name;

INSERT into tablename

order→ (id, age, name, email, follower, following)

values

(1, 2, "Papu", "abc", 24, 25);

SELECT COMMAND

Select and Show data from the DB

Syntax →

⇒ SELECT id, name, email FROM user;

⇒ SELECT * FROM user.

⇒ SELECT DISTINCT age FROM user;

Show unique age.

17 Oct 2026, 4:08 pm

Where Clause
(To define some condition.)

SELECT col1, col2 FROM table_name
Where condition.

Type of condition

Arithmetic : +, -, *, /, %

Comparison : =, !=, >, <, >=, <=

Logical : AND, OR, NOT, IN, BETWEEN, ALL, LIKE, ANY

BITWISE : & (Bitwise AND), | (Bitwise OR)

Logical →

AND → to check both the condition should be true

OR → to check one condition be true

BETWEEN → SELECT for a given range BETWEEN (17 AND 19);

IN → Match any value in list

NOT → to negate the given condition.

17/3/24
54

LIMIT Clause
Set an upper limit on number of (tuples) rows to be returned.

SELECT col1, col2 FORM table_name;
LIMIT number;

Order by Clause ASC DESC
To sort in ascending or descending order.

SELECT —
ORDER BY Column_name ASC/DESC;

Aggregate Function

Aggregate function perform a calculation on a set of value, and return a single value.

COUNT(), MAX(), MIN(), SUM(), AVG(), etc.

Eg. SELECT count(age)
From user
WHERE age = 14;

SELECT Max(follower)
From user;

17 Oct 2026, 4:09 pm

Group by clause

Groups rows that have the same values into summary row.

It collect data from multiple records and groups the result by one or more column.

☆ Generally we use group by with some aggregation function

```
SELECT age, max(followers)
From user
Group By age;
```

Aggregation function के अलावा जो भी column है उसे group by करना ही होगा।

17 Oct 2026, 4:09 pm

Having clause

Similar to where i.e. applise same condition on rows

But it is used when we want to apply any condition after grouping.

```
Select age, max(followers)
From user
Group by age,
Having max(followers > 200);
```

* Where is for the table, Having is for a group
* Grouping is necessary for Having.

General Order

```
SELECT      column(s)
FROM        table_name
WHERE       condition
GROUP BY    column(s)
HAVING      condition
ORDER BY    column(s) ASC, DESC;
```

17 Oct 2026, 4:09 pm

TABLE Queires

UPDATE

SET SQL_SAFE_UPDATES = 0;

```
Update   table_name
SET      follower = 600    // setting value for single user
Where    age = 16;         Specific column
```

DELETE (to DELETE Existing rows)

```
DELETE FROM table_name
Where       condition;
```

if we don't use where condition Delede Delede all the row from table.

ALTER TABLE Queiries.

to change in the schema

ADD Column
ALTER TABLE table name
ADD Column column_name datatype constraint;

DELETE Column

ALTER TABLE table name
DROP Column column_name;

Rename table.
ALTER TABLE user(tablename)
Rename To instaUser;

Change the column

ALTER —11—
CHANGE COLUMN old name newname datatype
Constraint;

17 Oct 2026, 4:09 pm

Date 18
Page 59

Modify column

ALTER -11-
MODIFY col_name newdatatype new constraint;

17 Oct 2026, 4:09 pm

Backend (5) (Node with SQL)

Using Faker

Just

(1) const {faker} = require ('@faker-js/faker');

(2) let ged_user = ( ) => {
    return {
        id: faker.string.uuid ();
        username: faker.internet.userName (),
        email : faker.internet.email ();
        password: faker.internet.password (),
        };
}

MYSQL2 Package

to connect Node with MYSQL.

// to close connection { connection.end () }

① consst mysql = require ('mysql2');

② Consst Conection = mysql.Creadeconnection ({
    host : "localhost",
    user : "root",
    database: "delta_app",
    password: "ParasMERN@12";
});

Using SQL from CLI

~~tusor~~ /usr/local/mysql/bin/mysql -u root -p

Not working.

4 way to use sql file

1. Workbench
2. MySQL pacage - node.js
3. CLI
4. Using sql file

mysql> USE database;
source schema.sql;

17 Oct 2026, 4:09 pm

Node with Sql.

After MYSQL 2 package.

```
try {
    connection.query("SHOW DATABASES", (err,
    result) => {
      if (err) throw err;
      console.log(result); // result is array.
    });
  } catch (err) {
      console.log(err);
    }
    connection.end();
```

इस "SHOW DATABASES" query को

अलग से भी लिख सकते हैं,

```
let q = "SHOW DATABASES";
```

से use कर सकते हैं।

17 Oct 2026, 4:09 pm

⇒ Create a user table
⇒ INSERT INTO user
⇒ INSERT in bulk.

⇒ Routing.

I take rest for few days

—X—

Now I am back.

(3/4/24)

Creating table using .sql file

Just Simple command in SQL file like in MySQL workbench.

Node + SQL.

create 9 = query then use q in my function

17 Oct 2026, 4:10 pm

Date 4/4/24
Page 65

INSERT INTO user

```
let q = "INSERT INTO user (id, username, email, pass)
        values (?, ?, ?, ?)";

let user = ["123", "123_user", "abc@gmail.com", "abc"]

try {
  conection.query(q, user, (err, result) => {

  }
conection.end();

// Single ? marks
// array of array = [user] in by. function
```

17 Oct 2026, 4:10 pm

INSERT IN BULK ⇒

Same as local just put data in Array and array insert in table.

Routing →

task → Get, Get, Patch, Post, Delete.

अब Connection end
दिखने की जरूरत नी when
res ponce send it automatic off.

7.1

req.q, req.params, req.body

↓ ↓ ↓ form से आया

link link

1.nk

Join

Inner JOIN	$A \cap B$
Left Join	$A \cup (A \cap B)$
Right Join	$B \cup (A \cap B)$
Full Join	$A \cup B$

27/5/24
7th

MongoDB →

Mongosh →

Some commands

cls	=	clear
help	=	help.
show dbs	=	show databases
use college	=	create db college
show collection	=	show collection.

Mongosh जैसा है वैसा ही हम JS Run कर सकते है, node shell पर

for eg. "apna college".toUpperCase()

जब हम mongosh run करते है तो temporary db create करता है named test.

17 Oct 2026, 4:10 pm

BSon Data
Binary JSon →

JSon	BSon
Data Support → String, Bool, Number Array, object, null	String Bool Num float long Decimal Bin data Array, date
Read Human Machine	Machine

—x—

Document & Collection

Doc - it store data

Collect → it store Document.

INSERT in DB →

db.student.insertOne({name: "Ayush", age: 15});

db.student.find() // show data

db.student.insertMany([{}, {}, {}])

17 Oct 2026, 4:11 pm

Find DB

db.collection.find() // → it return cursor
db.collection.findOne() // → it return doc.

db.collection.find({ city: Richhi 3})

condition

db.collection.find({ city: Richhi, mark: 75 3})

2 condition

Find in DB with operators.

← OPERATORS →

—x—

On mongodb side Resource reserve operator query

Comparison →

⇒ $eq, $gt, $gte, $in, $lt, $lte, $ne, $nin

logical

$and, $not, $nor, $or

17 Oct 2026, 4:11 pm

Implementation →

Query Ops.

Q→ find student where marks >75

A→ db.student.find ({marks: {$gt: 75}})

Q→ where city is delhi or mumbai

A→ db.student.find ({city: {$in: ["Delhi", "Mumbai"]}})

Q→ find student who scored >75 or live in Delhi

A→ db.student.find ({$or: [{marks: {$gt: 75}}, {city: "Delhi"}]})

17 Oct 2026, 4:11 pm

17 Oct 2026, 4:11 pm

UPDATE in DB →

update One -

db.collection.update(filter, update, options)

db.student.update({name:"adam"}, {$set {marks:999}})

⇒ Some more in DB - operator
$addfields, $set, $project, $unset, $replaceroot
$replace with

db.collection.updateOne()
db.collection.updateMany()
db.collection.replaceOne() → full doc replace

Nesting

db.student.insertOne({name:"xz", pro:{m:37, g:"A"}})

db.student.find({"pro.m":37});

DELETE

db.collection.deleteOne(filter, option)
db.collection.deleteMany(filter, option)
db.dropDatabase().

delete all
db.student.deleteMany({})

17 Oct 2026, 4:11 pm

← Mongo with Node.js →

① npm i mongoose

② const mongoose = require ("mongoose");

③ mongoose.connect ("mongodb://127.0.0.1:27017/Paras")

① →

② →

④
```
main()
.then ( () => {
    console.log ("connection succesful");
})
.catch ( (err) => console.log (err) });

async function main () {
    await mongoose.connect ("mongodb://127.0
                .0.1:27017/Paras");
}
```

5 Schema

Schema define the shape of the documents within that collection

```
Const userSchema = new mongoose.Schema ({
    name : String,
    email : String,
    age : Number,
});
```

⑥ Models

Model in mongoose is a class with which we construct documents.

```
const User = mongoose.model("User", UserSchema);
```

⑦ Insert

```
const user_1 = new User({
    name : "Adam",
    email : "ADM@gmail.com",
    age : 58,
});
```

#

but it will not show in db but it work and how check in command line

node

.load index.js

user_1

#

id automatic create from mongo

for save

#

```
user_1.save();   // one way
```

for more deep

Docs → Api → models → models.save()

Promises ची then() function use होतो.

24/1/24
79

// proper way

```
user_1
  .save()
  .then((res) => {
    console.log(res);
  })
  .catch((err) => {
    console.log(err);
  });
```

⑧ InSert Many

```
User.insertMany([
  { name: ... },
  { name: Paros, ... },
  { ... },
]).then((res) => {
  console.log(res);
})
```

⑨ Find

```
Model.find() // return a Query obj (thenable)
```

* Mongoose Queries are not promises. But they have a .then()

17 Oct 2026, 4:12 pm

```
User.find().then((data) => {
    console.log(data);
});
User.find({age: {$gte: 47 33}).then(data => {
    console.log(data);
}).catch((err => {
    console.log(err);
});
```

Same for findOne and findById

10 UPDATE

```
User.updateOne({name:Bruce" }, {age: 49})
  .then((res) => {
      console.log(res);
  })
  .catch ((err) => {
      console.log(err);
  });
```

```
User.updateMany({age: {$gt:48}}, {age: 55})
...
```

10.1 User.findOneAndUpdate({name:" "},
{age :423, {new: true})

17 Oct 2026, 4:12 pm

11 DELETE

User.deleteOne({ name: "Adam" }).then(...);

User.deleteMany({ age: { $gt: 40 } }).then(...);

Model.findOneAndDelete()
Model.findByIdAndDelete()

12 # Schema Validations

Basically Rules for Schema.

```
Const bookSchema = new mongoose.Schema({
    title: {
        type: String,
        required: true,
    },
    Price: {
        type: Number,
    }
});
```

17 Oct 2026, 4:12 pm

17 Oct 2026, 4:12 pm

12.1 Options

required : true, default : 0

Select :

Validate
get :
alise :
transform :
genre : string → genre : ["", "", ""] store strings array

for String

lowercase : uppercase :
trim :
min length : max length :
enum : array [red, green, ...] if same ok otherwise no.

for Number

min : 1 max :
↓
no less then :1

enum : populate :

for Date

min
max expaires :

13 Updation & Error

Schema validation update के समय काम नहीं करता।

Book.findByIdAndUpdate("id", {price: -1003});

⇒ done without error, even price issued is min) for id

("Id", {price:-1003, ~~runValidators~~ runValidators: true});

Error

Vali⇒ min: [1, "Price is too low"];

in .catch((err) ⇒ {
console.log(err.errors.price.properties.message);
});

17 Oct 2026, 4:12 pm

=> Mongo with Express. <=

we create a little program for understand it.

Step.0 Npm init -y, npm i ejs, npm i express
npm i mongoose.

Step 1

```
Const express = require ("express");
Const App = express();
let port = 8080;
const path = require ("path");
Const mongoose = require ("mongoose");
const methodOverride = require("method-override");
```

```
app.set ("view engine", "ejs");
app.set ("views", path.join (__dirname, "views"));
```

```
app.use (express.urlencoded ({ extended: true }));
app.use (express.static ('Public'));
app.use (express.static (path.join (__dirname, "public")));
app.use (methodOverride ("_method"));
```

17 Oct 2026, 4:12 pm

17 Oct 2026, 4:12 pm

```
const Chat = require("./models/chat.js"); // not common

main()
.then(()=>{
    console.log("you are connected");
})
.catch((err)=>{
    console.log(err);
});

async function main(){
    await mongoose.connect("mongodb---");
}

app.get("/",(req,res)=>{
    res.render("Home.ejs");});

app.listen(port,()=>{
    console.log("listening to port:8080");
});
```

common for